

Week 5: Shading

Determining Surface Details and maybe Better Rendering Quality

CS 4212: Computer Graphics

Fall 2026

Shading, Light Reflectance, Sampling

Once you hit a Shape, what color does it reflect back?

- Tue - Shading basics, scene organization,
- Lab - Implement some Shaders: Lambertian & Blinn-Phong
- Thu - Expand on Ideas of Shading

In-Class Review

- What are the reasons we include a minimum and maximum t when checking intersections and in what ways does it help your search?
- What can the dot product tell you about the relationship between how two vectors are oriented relative to each other?
- Why is it better for our intersect function to return a bool and rather than just returning the color of the shape?
- How does our software architecture allow us to render a scene containing both spheres and triangles in the same loop? Why does this matter?
- If your triangle code returns $\alpha = 0.4$, $\beta = 0.7$, and $\gamma = 0.1$, what should your code return?
- Conceptually, what is important when computing an actual β , or γ or α parameter?

Next Steps after Ray/Shape Intersection

Multiple Shapes!

- Need some notion of a “Scene”
- Book’s approach is not bad either (*hittable list*)

In-Class Exercise

Working with partner, sketch out the Scene Class design.

What do we need to plan for?

Next Steps after Ray/Shape Intersection

When you intersect a shape, what can you do?

- Turn the pixel on if hit, use bg color otherwise (hit shader)

Next Steps after Ray/Shape Intersection

When you intersect a shape, what can you do?

- Turn the pixel on if hit, use bg color otherwise (hit shader)
- Shade it? What options do we have for shading?

In-Class Exercise

What happens when a ray misses? What should you do? What does the book do?

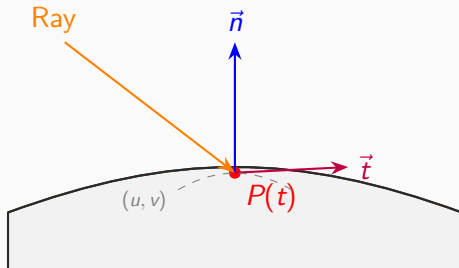
What do you think of the interval class from the book?

What do we know about the surface?

The Hit Record: When a ray hits an object, we need more than just its " t " parameter. We need to know:

- **Position (P):** Where did we hit?
- **Normal ($\vec{n}$):** "up" on the surface
- **UV Coordinates (u, v):** Texture coordinate
- **Tangent ($\vec{t}$):** Vector along the surface

These properties allow us to compute lighting, shadows, and eventually look up textures (what would textures provide us?).



The Hit Structure

Important to know *what* was hit, *how* it was hit, and *where* it was hit!

- Can be encoded in class or structure (*HitStructure*)
- Contains what you need for shading operations, some ideas follow:
 - t
 - Normal vector at point of intersection
 - Pointer to object that was intersected
 - Pointer to shader that should be used

```
im  $\leftarrow$  new image(  $n_x n_y * 3$  )
for all  $p_{i,j} \in \text{im}$  do
  {Compute Viewing Ray}
   $u = l + (r - l)(i + 0.5)/n_x$ 
   $v = b + (t - b)(j + 0.5)/n_y$ 
  Ray  $r$ ;
   $r.\text{direction} = -d\vec{W} + u\vec{U} + v\vec{V}$ ;
   $r.\text{origin} = \vec{e}$ ;
  {Intersect with Objects}
  HitStructure  $h$ ;
  for all  $o_k \in \text{objectList}$  do
     $o_k \rightarrow \text{intersect}( r, t_{min}, t_{max}, \&h )$ 
  end for
  {Set pixel color based on object shader}
   $\text{im}[p_{i,j}] \leftarrow h \rightarrow \text{applyShader}(o_{k_{closest}})$ 
end for
```

Why do we need this?

1. **Shading ($\vec{n}$):** Light interacts with surface orientation. (e.g. $\vec{n} \cdot \vec{l}$)
2. **Texture Mapping (u, v):** "Painting" images onto shapes.
3. **Bump Mapping:** Checking local perturbations to $\vec{n}$.
4. **Secondary Rays:** Starting point P for reflection/refraction.

Without this data, we just have a silhouette (or a hit shader)!

Computing Normals

Primitive 1: Sphere Normals

Geometry:

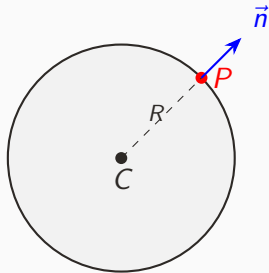
- Defined by Center C and Radius R .
- A point P is on the surface if $|P - C| = R$.

The Normal

The normal is simply the direction from the Center to the point.

$$\vec{n} = \text{normalize}(P - C)$$

* Remember to normalize! The magnitude of $(P - C)$ is R .



Primitive 2: Triangle Normals

Geometry:

- Defined by 3 vertices: A, B, C .
- The face is planar (flat).

The Normal

Computed via the **Cross Product** of two edges.

$$\vec{u} = B - A$$

$$\vec{v} = C - A$$

$$\vec{n} = \text{normalize}(\vec{u} \times \vec{v})$$

What happens if you specify the A, B and C,
vs. A, C, B?

Primitive 2: Triangle Normals

Geometry:

- Defined by 3 vertices: A, B, C .
- The face is planar (flat).

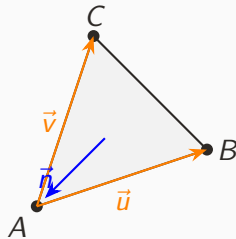
The Normal

Computed via the **Cross Product** of two edges.

$$\vec{u} = B - A$$

$$\vec{v} = C - A$$

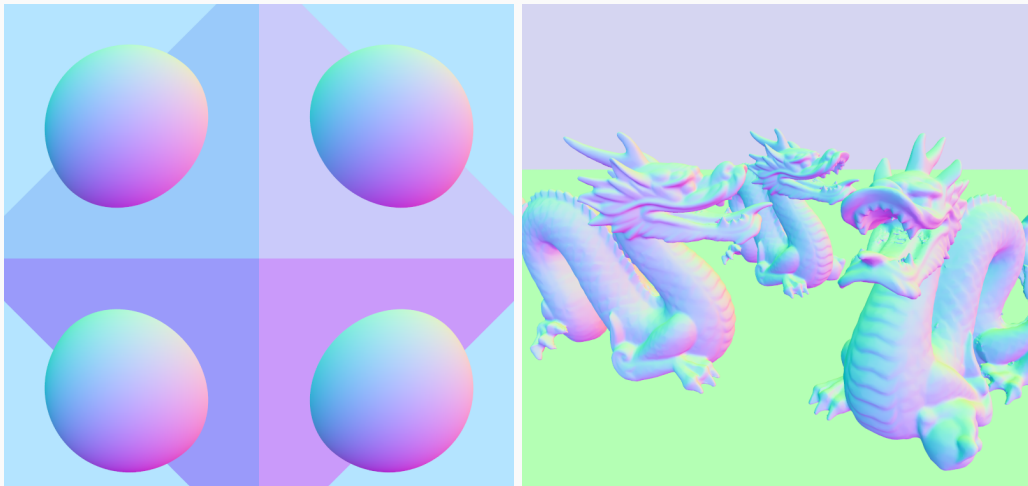
$$\vec{n} = \text{normalize}(\vec{u} \times \vec{v})$$



What happens if you specify the A, B and C, vs. A, C, B? * Direction depends on winding order (Counter-Clockwise).

Shading - Normal Map

What can you do with normal information? Use your Half-Space Mapping and make a Normal Shader!



Normal Shader

Without looking at notes, what range is normal vector in?

Write down how you convert it to a color.

Why are these normal shader images so pastel and blue'ish?



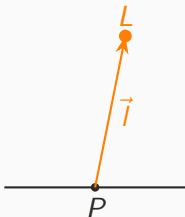
Light Sources

Light Sources: Point vs. Area

We need the notion of a light for some of our shaders...

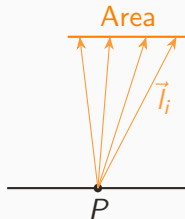
1. Point Light

- Infinitesimal point in space (not reality)
- **One** shadow ray per surface point
- Result: **Hard Shadows**.



2. Area Light - for later

- Physical size (e.g., LED panel).
- **Multiple** shadow rays (sampling).
- Result: **Soft Shadows**.



Many Ways to Shade an Shape!

- Direct! Older style shaders: Lambertian & Blinn-Phong
- Indirect, Diffuse - use surface properties to scatter light!

Direct Illumination Models

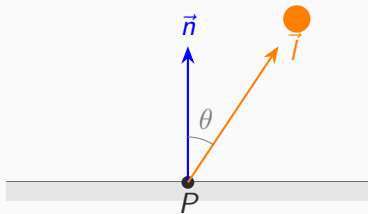
Lambertian Reflectance

- Surfaces reflect light equally in all directions.
- Brightness depends on the **angle** of incidence.
- Modeled by **Johann Lambert** (1760, Photometria).

Lambert's Law

$$I \propto \max(0, \vec{n} \cdot \vec{l}) = \cos \theta$$

If light hits perpendicularly ($\theta = 0^\circ$), it's brightest. If it hits edge-on ($\theta = 90^\circ$), it's black.



Blinn-Phong Reflectance - Captures the Specular Highlight

- Captures the "shininess" of a surface.
- Based on the **half-vector** $\vec{h}$.
- $\vec{h} = \text{normalize}(\vec{l} + \vec{v})$.

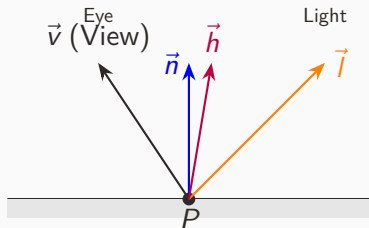
The Formula

$$I_{spec} \propto (\vec{n} \cdot \vec{h})^p$$

The Phong Exponent (p):

- Controls the *sharpness* of the highlight.
- Higher p (e.g., 100) $\rightarrow$ Smaller, sharper spot.
- Lower p (e.g., 10) $\rightarrow$ Broad, dull spot.

How does the half-vector represent real surfaces and why?



Wednesday Lab

Goals for Lab - Write a Shader

- Add Shader and Hit Structure
- Implement the Lambertian and Blinn-Phong Shaders

Shading - Lambertian

Simplest model - based on Lambert's Law

- Amount of energy falling on surface is proportional to the angle of the surface to the light

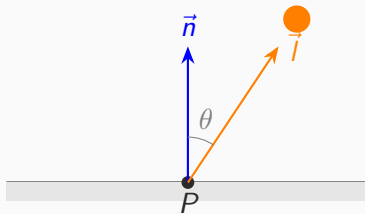
Think about the cosine function and the dot product!

- $\cos(0) = 1$, $\cos(90) = 0$, etc...

$$L = k_d I_{light} \cos \theta$$

$$L = k_d I_{light} \max(0, \vec{n} \cdot \vec{light}_{dir})$$

- L is the intensity of the energy or light falling on the surface
- k_d is the diffuse reflectance of the surface
- I_{light} is the intensity of the light source

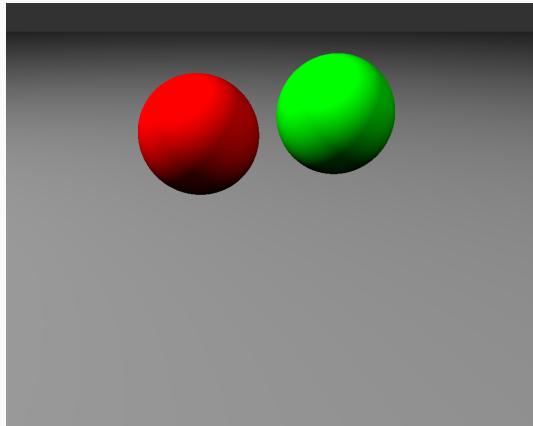


Shading - All the Lights

Shading model must support multiple lights

- Simple in that all shading values from each light are summed together
- Easy to get values greater than 1, so intensities may need to be tweaked
- You may need to clamp values within $[0, 1]$ range!

$$L = \sum_{i=1}^N k_d I_{light_i} \max(0, \vec{n} \cdot \vec{light}_{i_{dir}})$$

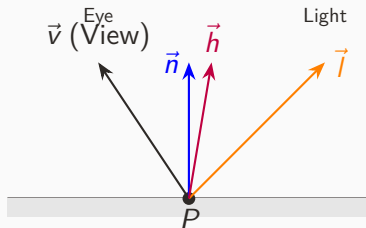


Shading - Specularity

- Not all surfaces are matte (Lambertian)!
- Many have specular highlights or reflections
 - What is a specular highlight?

Shading - Specularity

- Not all surfaces are matte (Lambertian)!
- Many have specular highlights or reflections
 - What is a specular highlight?
 - Your eye observing the reflected light on the object!
- Simple specular shading model created by Phong (1975) and updated by Blinn (1976)
- Still commonly used!
- *Basic idea*: reflection of light source is brightest when $\vec{l}$ and $\vec{v}$ are symmetric about $\vec{n}$, and hence, the reflection vector $\vec{r}$ coincides with $\vec{v}$



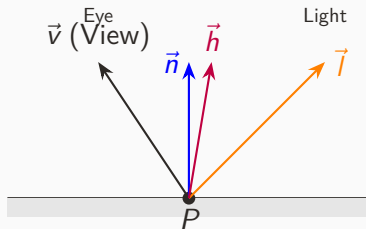
Shading - Blinn Phong Model

Observation from Jim Blinn in 1977 that different model using the half-vector more accurately represents specular hi-lights:

- $\vec{h}$ is bisector between $\vec{l}$ and $\vec{v}$
- When $\vec{h}$ coincident with $\vec{n}$, we have bright specular highlight

$$\vec{h} = \frac{\vec{v} + \vec{l}}{\|\vec{v} + \vec{l}\|}$$

$$L = k_d I \max(0, \vec{n} \cdot \vec{l}) + k_s I \max(0, \vec{n} \cdot \vec{h})^p$$



Thursday

Goals for the Rest of The Week

- Work to generalize the rendering core.
- Make sure shader concepts work and are understood.
- Some new features you could implement?
- Heading towards Multisample Antialiasing
- Maybe... More Realistic Diffuse Shading but maybe next week.

What is diffuse reflectance?

What is the amount of energy a surface receives?

What is the energy that is reflected back to a viewer??

Lambertian (Diffuse)

- How does it approximate diffuse reflectance?

Blinn-Phong (Specular)

- How does it approximate specularity?

Lambertian (Diffuse)

- How does it approximate diffuse reflectance?
- Depends only on angle between $\vec{n}$ and $\vec{l}$.
- $I \propto \max(0, \vec{n} \cdot \vec{l})$
- Models light scattering in all directions. What does this mean?

Blinn-Phong (Specular)

- How does it approximate specularity?

Lambertian (Diffuse)

- How does it approximate diffuse reflectance?
- Depends only on angle between $\vec{n}$ and $\vec{l}$.
- $I \propto \max(0, \vec{n} \cdot \vec{l})$
- Models light scattering in all directions. What does this mean?

Blinn-Phong (Specular)

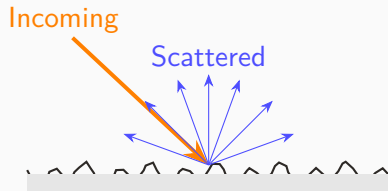
- How does it approximate specularity?
- Depends on half-vector $\vec{h}$ and normal $\vec{n}$.
- $I \propto (\vec{n} \cdot \vec{h})^p$
- p controls "shininess" (tightness of highlight).

Micro-Geometry: Diffuse Scattering

What is "Diffuse"?

- Physically, these surfaces are rough at a microscopic level.
- An incoming light ray hits a micro-facet and bounces.
- For a **perfectly diffuse** (Lambertian) surface, these micro-bounces randomize the outgoing direction.

Result: Light is scattered over the entire hemisphere. The surface looks the same brightness from any viewing angle.



Diffuse Surfaces: The Viewer (View Independence)

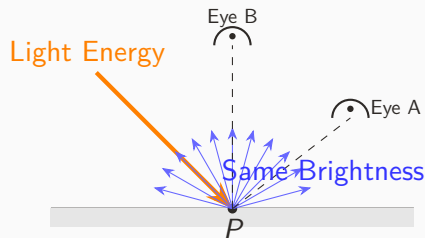
Concept: Once light hits a diffuse surface, it scatters **equally** in all directions.

- The surface brightness looks the **same** from any angle.
- Brightness does **not** depend on the view vector $\vec{v}$.

Light Reflected

$$L_{out}(\vec{v}) = \text{Constant}$$

(Given fixed lighting direction and energy)



The Light Source (tied to the Cosine Law)

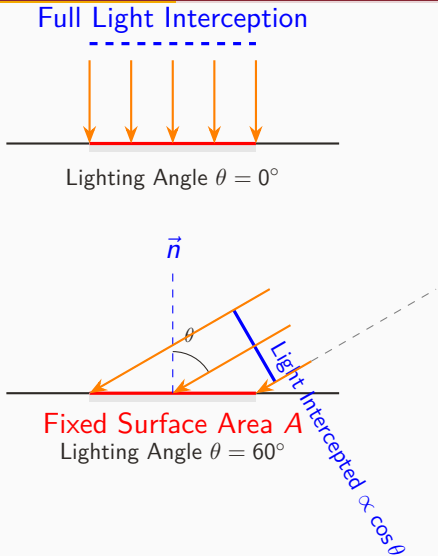
Concept: The amount of light energy a surface receives depends on the angle of the light.

- A fixed surface area A intercepts less light as the angle increases.
- In other words, the "Projected Area" facing the light shrinks.

Light Received

Received Light $\propto \cos \theta$

Intensity $\propto \vec{n} \cdot \vec{l}$



What does the Light See?

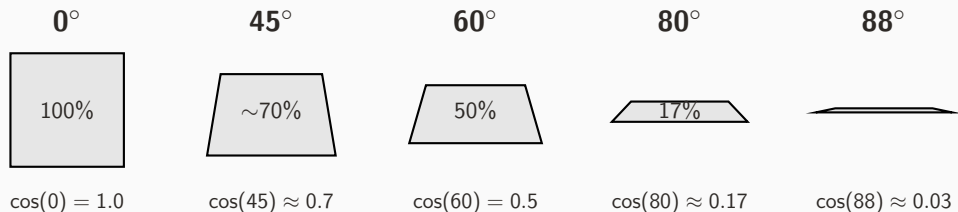
Imagine a small square patch of area A . As the angle between the light and the normal goes from 0 to 90, what does the light see?

Draw the patch?

The Light's Perspective: Foreshortening

How much surface does the light see? Imagine looking down the light ray at a square surface patch.

As the angle θ increases, the patch appears "squashed" (foreshortened).



At 90° (**Grazing Angle**), the projected area becomes **zero**.

No light is intercepted $\rightarrow$ Black.

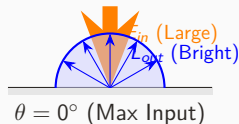
Energy Balance: From Incoming to Outgoing

Two Different Rules Apply:

1. Incoming (Angle Matters)

The energy **received** depends on the light angle θ .

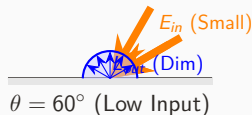
$$E_{in} \propto I_L \cos \theta$$



2. Outgoing (View Independent)

The energy **reflected** is a fractional amount of (k_d) of what was received, scattered equally.

$$L_{out} \propto k_d E_{in}$$

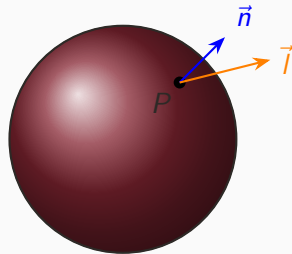


Putting it Together: The Lambertian Model

The complete model:

$$L_{diffuse} = k_d I_L (\vec{n} \cdot \vec{l})$$

1. **Light Interaction ($\vec{n} \cdot \vec{l}$):** Determines how much light energy hits the surface.
2. **Surface Material (k_d):** Determines the color reflected.
3. **Light Color (I_l):** Determines the color of light energy.
4. **View Independence:** Reflects equally in all directions.



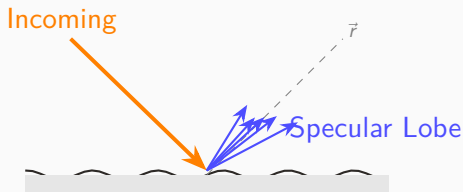
Result: Smooth shading

Micro-Geometry: Glossy Reflection (Blinn-Phong)

What is "Glossy"?

- Surfaces are **mostly smooth** but have small micro-facet variations.
- Light bounces off facets that are **nearly aligned** with the main surface.
- Reflection forms a **lobe** around the perfect reflection direction $\vec{r}$.

Result: A blurred, bright highlight. The highlight is visible only when the viewer is near the reflection angle.



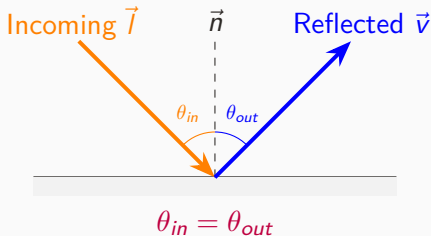
Micro-Geometry: Perfect Mirror (Ideal Specular)

What is a "Mirror"?

- Surfaces are **perfectly smooth** at the microscopic level.
- No random micro-facets.
- **Law of Reflection:**

$$\theta_{in} = \theta_{out}$$

Result: A sharp, undistorted reflection.
Light travels in a **single** outgoing direction for each incoming ray.



Approximation vs. Reality

Important Note

Lambertian and Phong models are **empirical approximations**.

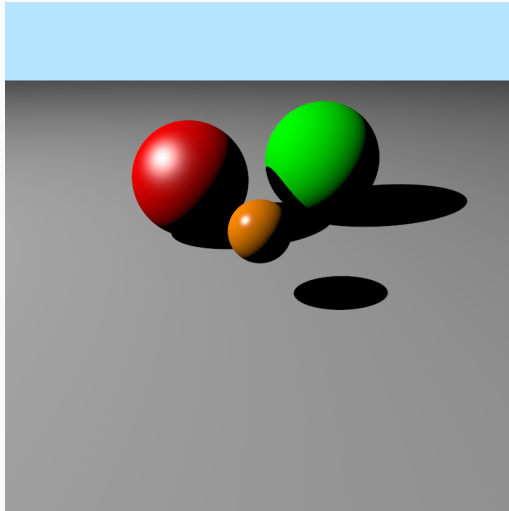
Why learn them?

- **Historical Context:** They defined real-time graphics for decades.
- **Efficiency:** Extremely cheap to compute.
- **Intuition:** They introduce vectors $(\vec{n}, \vec{l}, \vec{v}, \vec{h})$ used in complex models.

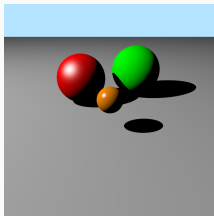
The Future (Next Lectures):

- **Physically Based Rendering (PBR):** Enforcing energy conservation.
- **Microfacet Theory:** Statistical modeling of surface roughness.
- **Fresnel Effects:** Light reflection varies by angle.

How can you do this?

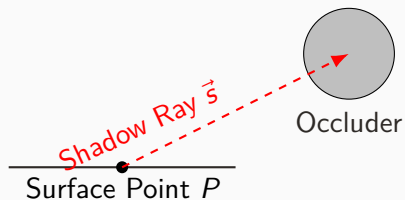


Casting Shadows



- Does the Local Illumination model account for objects blocking light?
- **Question:** What change do we need to make to our rendering code to support shadows?
- *Discuss with your neighbor.*

The Shadow Test



Light Source L

1. Construct Ray $\vec{r}(t) = P + t(L - P)$
2. Check scene intersection for $t \in [\epsilon, 1]$
3. If hit:
In Shadow (Color = Ambient)
4. Else:
Lit (Add Diffuse + Specular)

Anti-Aliasing & Sampling

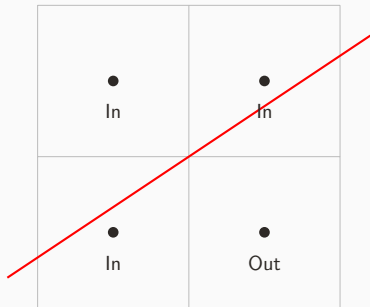
The Problem: Aliasing

"Jaggies" and Artifacts

- A pixel is a square area, not a single point.
- Sampling at just the **center** (0.5, 0.5) misses detail.
- High-frequency features (sharp edges, thin shadows) get lost or "jagged".

The Solution

Super-Sampling: Cast multiple rays per pixel and average the result.



Center sampling misses the coverage ratio.

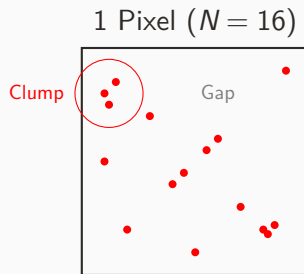
Pure Random Sampling (Monte Carlo)

The Algorithm:

- Pick N random points (x_i, y_i) inside the pixel.
- Average the color: $C = \frac{1}{N} \sum C_i$.

Pros & Cons:

- + Easy to implement.
- + Turns "jaggies" into noise (better for eyes).
- **Clumping:** Random points bunch up.
- **Gaps:** Some areas are missed.



Stratified (Jittered) Sampling

The Improvement:

- Divide pixel into an $N \times N$ grid (strata).
- Pick **one** random point per cell.

Why it helps:

- **No Clumping:** Points are forced apart.
- **No Large Gaps:** Every corner counts.
- Converges to the true image **faster** (requires fewer rays for same quality).

4x4 Grid ($N = 16$)

